

目次

第 10 章	グループワーク（前半）	2
10.1	複数のファイルにまたがるプログラム	4
10.2	ライブラリの共有	5
10.3	全体のファイル構成	7
10.4	ユニットテスト	9
索引		14

第 10 章

グループワーク（前半）

ここでは複雑で不明確な課題に取り組むことを通して、グループで比較的大きなプログラムを作成する。具体的には、「短いテキストから筆者の感情を識別するプログラム」を Python を用いて作成し、UTOL 上のテキストファイルの各行がどのような感情かを判別する。テキストファイルは以下のような文からなる。

data.txt の最初の 8 行

```
1 朝の空がきれいで、気持ちよく一日を始められた。
2 寝坊して電車に乗り遅れ、朝から気分が沈んだ。
3 朝食にはトーストとヨーグルトを食べた。
4 提出したレポートを褒められて、少し自信がついた。
5 準備していた資料を忘れてしまい、ひどく焦った。
6 駅前のスーパーは午後九時まで営業している。
7 久しぶりに友人と会えて、話が尽きなかった。
8 友人との約束が急にキャンセルになって、少し寂しかった。
```

「テキストから感情を識別する」というだけでは問題は正確に定義されていないので、各グループで問

題（タスク）を定義して、作成するプログラムの要件を明確にしていく。ここでは単純に感情を「ポジティブ・ネガティブ・ニュートラル」に大別してそれらの分類を行う問題として定義する。このような問題は自然言語処理において**極性分類**とよばれる。極性分類の解法は2つのアプローチに大別される。一つの方法は極性辞書を用いてルールを人手で作成するアプローチであり、もう一つは教師データと呼ばれるデータセットから**予測器**を学習する、機械学習手法 ([Scikit-learn](#)などのライブラリ) を利用するアプローチである。後者のアプローチの方が高精度なプログラムが作れることが知られているが、前者の方法論よりも複雑なのでグループワークの後半（第三週以降）で行う。

ここでは東北大学自然言語処理研究グループが提供している[日本語評価極性辞書](#)を用いることを想定している。辞書は2つに分かれており、UTOLにてこれらのデータを dictionary1.txt,dictionary2.txt という名前でダウンロード可能である。

日本語評価極性辞書の内容（一部）

1	二度寝	e	～する（行為）
2	二日	e	～である・になる（状態）客観
3	二日酔い	n	～である・になる（状態）客観
4	二年半	e	～である・になる（状態）客観
5	二枚舌	n	～である・になる（評価・感情）主観
6	二枚目	p	～である・になる（評価・感情）主観
7	二律背反	n	～する（出来事）
8	二流	n	～である・になる（評価・感情）主観
9	賑やか	p	～である・になる（評価・感情）主観
10	賑やかさ	p	～がある・高まる（存在・性質）

ここでeはニュートラル,nはネガティブ,pはポジティブの意味である。以降ではこのデータを利用することを想定しているが、実際にどのような辞書をどのように使うかはグループで決定する。

10.1 複数のファイルにまたがるプログラム

まず、比較的大きなプログラムを構成するための方法について説明する。ここまでの講義ではプログラムは一つのファイルに書いて提出していたが、複雑なプログラムが複数のファイルにまたがることは普通である。ライブラリをインポートするために導入した `import` 文を用いて、別のファイルで定義された関数やクラスを用いることができる。ただし、そのファイルをインタプリタが特定できるように、ファイルを適当な場所に置くか、環境変数を設定する必要がある。例えば、以下のようなファイル構成が用意されているとする。

ファイル構成例

```
1 project_root/  
2 |— main.py  
3 |— src/  
4 |   |— bad_main.py  
5 |— some_library/  
6 |   |— good_class.py  
7 |   |— nice_class.py
```

この時、以下のようにすると、`good_class.py` ファイルの中で定義されたクラスや関数を `main.py` から呼び出すことができる^{*1}。

^{*1} インポートする対象は他の `python` のプログラムファイルである場合もあれば、コンパイル済みのバイナリコードを表すファイルである場合や実態が存在しない場合などもあり様々である（詳細は省略する）。

main.py の内容 (bad_main.py も内容は同じ)

```
1 import some_library.good_class
2
3 my_good_instance = some_library.good_class.GoodClass() #インポートしたクラスを利用する
4 some_library.good_class.good_func() #インポートした関数を利用する
```

good_class.py の内容

```
1 class GoodClass:
2     def __init__(self,a):
3         self.value = a
4         print("Good class!")
5 def good_func():
6     print("Good function!")
```

(ターミナルでの) 標準モードの実行例

```
1 $ python3 main.py
2 Good class!
3 Good function!
4 $ python3 src/bad_main.py
5 Traceback (most recent call last):
6   File "src/bad_main.py", line 1, in <module>
7     import some_library.good_class
8 ModuleNotFoundError: No module named 'some_library'
```

第一行目は `some_library/good_class.py` をモジュールとしてインポートすることを意味する。import 文でディレクトリの間は「/」ではなく「.」で区切られており、「.py」が省略されていることに注意する。`bad_main.py` は直下に `some_library` ディレクトリがないためインポートに失敗する。

10.2 ライブラリの共有

PyPI を用いて環境にインストールするライブラリは `requirements.txt` に記述しておくのが慣例である。

requirements.txt の内容例

```
1 numpy == 1.20.0 #前 述
2 pytest == 8.2.2 #ユニットテストのためのライブラリ
3 scikit-learn == 0.24.1 #機械学習のライブラリ
4 zstandard # 高速なデータ圧縮・解凍ライブラリ
5 janome # 日本語形態素解析のライブラリ
6 logzero # ロギングのライブラリ
7 chardet # 文字エンコーディングの自動検出ライブラリ
```

「==」以降はバージョンを指定している。下記のようなコマンドを用いると requirements.txt に記述されている PyPI のパッケージを一括でインストールすることができる。

(ターミナルでの) 標準モードの実行例

```
1 $ pip3 install -r requirements.txt
```

PyPI には登録されていないようなライブラリが Git のレポジトリとして公開されている場合、(git コマンドが使える環境であれば) 以下のコマンドを使用することで指定したレポジトリがワーキングディレクトリ直下に作成される。これら自作するプログラムから利用するためには上述の事情を踏まえて適切に import 文を使う。

(ターミナルでの) 標準モードの実行例

```
1 $ git clone <レポジトリがある場所のURL>
2 Cloning into '<レポジトリ名>'...
3 remote: Enumerating objects: 123, done.
4 remote: Counting objects: 100% (123/123), done.
5 remote: Compressing objects: 100% (45/45), done.
6 Receiving objects: 100% (123/123), 67.89 KiB | 123.45 MiB/s, done.
7 Resolving deltas: 100% (67/67), done.
```

10.3 全体のファイル構成

プログラムは複数のファイルにまたがるだけでなく、上記のようにしてインストールするライブラリなども存在する。グループで一つのプログラムの全体像を完成させるために、ファイル構成の全体像が分かるように整理して共有する。以下は全体のファイル構成の一例である。

プログラムのファイル構成例

```
1 my_nlp_project/
2   └─ src/
3       └─ main.py # メインの実行プログラム
4       └─ preprocess_data.py # データの前処理を行うプログラム
5       └─ my_library/
6           └─ load_input_data.py # 入力データを読み込むモジュール
7           └─ load_dictionary.py # 極性辞書を読み込むモジュール
8           └─ count_polarity_statistics.py # 単語の極性情報を計算するモジュール
9           └─ predict_polarity.py # テキストの極性を予測するモジュール
10          └─ manage_output.py # 出力を管理するモジュール
11   └─ data/
12       └─ dictionary1.txt
13       └─ dictionary2.txt
14       └─ data.txt # 入力データファイル
15       └─ processed_data.txt # 処理済みデータファイル
16   └─ logs/ # ログファイルの出力先ディレクトリ
17   └─ tests/
18       └─ test_preprocess.py
19       └─ test_load_input_data.py
20       └─ test_load_dictionary.py
21       └─ test_count_polarity_statistics.py
22       └─ test_predict_polarity.py
23       └─ test_manage_output.py
24   └─ requirements.txt
25   └─ README.md # プロジェクトの説明
```

例えば main.py の内容は以下のようなものを想定しているが、実際のプログラムは各グループで決定する。

main.py の内容例

```
1 from my_library import load_input_data as input_loader
2 from my_library import load_dictionary as dictionary_loader
3 from my_library import count_polarity_statistics as counter
4 from my_library import predict_polarity as predictor
5 from my_library import manage_output as output_manager
6
7 sentence_arrays = input_loader.load("data/processed_data_v1.txt")
8
9 d1 = dictionary_loader.load("data/dictionary1.txt")
10 d2 = dictionary_loader.load("data/dictionary2.txt")
11
12 result = []
13 for i in range(len(sentence_arrays)):
14     sentence = sentence_arrays[i]
15     count_statistics = counter.count(d1,d2,sentence)
16     result.append( predictor.predict(count_statistics) )
17
18 output_manager.output(sentence_arrays, result)
```

一行目は”src/my_library” 以下の”load_input_data.py” をインポートして input_loader というエイリアスを利用することを意味している。このプログラムは、プロジェクト直下 my_nlp_project/ で以下の様に実行することを想定している。

（ターミナルでの）標準モードの実行例

```
1 $ python3 src/main.py
```

これを実行しても、カレントディレクトリが src/ に移動するわけではない。プログラム中の data/... という相対パスは、コマンドを実行したプロジェクト直下からの相対パスである。

10.4 ユニットテスト

ユニットテストとは切り分けた個々の機能が正しく実装されているかを別々に検証することや、その個別の検証のことをいう。例えば上記の例でいえば（`predict_polarity.py` の具体的な内容はあえて説明はしないが、）`predict_polarity.py` だけをいくつかの代表的な入力に対して期待される出力が得られるかを検証する。python でユニットテスト行うには `pytest` ライブラリを用いることで簡単にできる。以下のようにプログラムを作成し、`pytest` コマンドを実行すればよい。詳しくは `pytest` のリファレンスを参照すること。

テストプログラムは `tests/` 以下におき、テスト対象のプログラムは `src/` 以下にある。そのままでは `tests/` から `src/` 以下のモジュールを `import` できないため、ここでは簡便な方法として `sys.path` を変更する方法を用いる^{*2}。

^{*2} 実際のプロジェクトでは、`pytest` の設定ファイルを変更したりすることも多い

test_predict_polarity.py の作成例

```
1 import os
2 import sys
3 project_root_path = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
4 sys.path.append(project_root_path)
5 import src.my_library.predict_polarity as predict_polarity
6
7 def test_predict_positive():
8     # ポジティブな入力に対する予測結果をテスト
9     input_statistics = [5, 1]
10    expected_output = 'positive'
11    assert predict_polarity.predict(input_statistics) == expected_output
12
13 def test_predict_negative():
14     # ネガティブな入力に対する予測結果をテスト
15     input_statistics = [1, 4]
16     expected_output = 'negative'
17     assert predict_polarity.predict(input_statistics) == expected_output
18
19 def test_predict_neutral():
20     # 中立的な入力に対する予測結果をテスト
21     input_statistics = [3, 3]
22     expected_output = 'neutral'
23     assert predict_polarity.predict(input_statistics) == expected_output
24
25 def test_predict_empty():
26     # 空リストの入力に対する予測結果をテスト
27     input_statistics = []
28     expected_output = 'neutral'
29     assert predict_polarity.predict(input_statistics) == expected_output
```

`__file__` はファイルのある場所が文字列として格納されている特別な変数であり、`os.path.abspath(__file__)` でその絶対パスが取得できる。結果として `project_root_path` を指す場所が `sys.path` に追加されるため、直下からのパスでライブラリの場所を指定できる。

`pytest tests/test_predict_polarity.py` などとすればユニットテストを実行できる。

pytest による上記プログラムの使用例（predict_polarity.py にエラーがある場合）

```

1 $ pytest tests/test_predict_polarity.py
2 ===== test session starts =====
3 platform linux -- Python 3.8.10, pytest-8.2.2, pluggy-1.5.0
4 rootdir: /home/nantoka/my_nlp_project
5 collected 4 items
6
7 tests/test_predict_polarity.py ...F [100%]
8
9 ===== FAILURES =====
10 _____ test_predict_empty _____
11
12     def test_predict_empty():
13         # 空リストの入力に対する予測結果をテスト
14         input_statistics = []
15         expected_output = 'neutral'
16     > assert predict_polarity.predict(input_statistics) == expected_output
17
18 tests/test_predict_polarity.py:31:
19 -----
20
21 count_stat = []
22
23     def predict(count_stat):
24     >     if count_stat[0] < count_stat[1]:
25 E         IndexError: list index out of range
26
27 src/my_library/predict_polarity.py:2: IndexError
28 ===== short test summary info =====
29 FAILED tests/test_predict_polarity.py::test_predict_empty - IndexError: list index out of range
30 ===== 1 failed, 3 passed in 0.12s =====

```

7 行目の”...F” は最初の 3 つのテストは正しく終了し、最後のテストに失敗したことを示している。これらのテストがすべて成功した場合は以下の様な出力を得る：

pytest による上記プログラムの使用例（predict_polarity.py のエラーを修正した場合）

```
1 ===== test session starts =====
2 platform linux -- Python 3.8.10, pytest-8.2.2, pluggy-1.5.0
3 rootdir: /home/nantoka/my_nlp_project
4 collected 4 items
5
6 tests/test_predict_polarity.py ....
7
8 ===== 4 passed in 0.09s =====
```

演習

グループごとに適宜利用可能なライブラリや文献を調査しながら、以下の 2 つの課題を提出すること。
グループ内のコミュニケーション（ファイル共有など）は slack のチャンネルを利用してください。

グループワーク：課題 1（一週目まで）

UTOL で配布されているファイルをダウンロードして、上述のプログラムのファイル構成例に従い個別の環境（ECCS 端末でもそのほかの PC でもよい）にプロジェクトを作成する。配布されていない部分については自分でファイルを用意する。ファイルの中身は動作するプログラムがなくてもよい。さらにグループでどのような関数を作る必要がありそうか、どのようなユニットテストを用意するべきかを相談し 1. 途中で困った個所とその解決方法、2. 現状決まったこと、3. 感想、を**各自**で報告する。

グループワーク：課題 2（二週目まで）

クラス/ファイル単位で管理の担当を割り当てて、各自が実装したものを合わせて、プログラムを実装・実行し、1. プログラムの構成の計画、2. だれが何を行ったか、3. 自身が課題に取り組んで途中で困った個所とその解決方法、4. 出力結果の評価、5. 感想をレポート形式にして**各自**で報告する。**レポートにはグループ全員の氏名・学籍番号を明記すること。**

参考

1. 日本語評価極性辞書，東北大学自然言語処理研究グループ. <https://www.nlp.ecei.tohoku.ac.jp/research/open-resources/> 2026/06/18 現在
2. pytest, <https://docs.pytest.org/> 2026/06/18 現在

索引

極性分類, [3](#)

ユニットテスト, [9](#)

予測器, [3](#)